



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Программное обеспечение ЭВМ и информационные технологии

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7
по курсу «Экономика программной инженерии»

«Оценка параметров программного проекта
с использованием модели СОСОМО II»

Студент Пермякова Екатерина Дмитриевна
Поляков Андрей Игоревич

Группа ИУ7 – 82Б

Преподаватель Барышникова Марина Юрьевна

2026 г.

Метод функциональных точек

Функциональная точка – это единица измерения, выражающая объем бизнес-функциональности, которую программная система предоставляет пользователю.

Определение числа функциональных точек является методом количественной оценки размера и сложности программной системы с точки зрения функций, которые система предоставляет пользователю.

Функциональные типы данных и типы транзакций, используемые в методе функциональных точек:

- **Внешний ввод** (EI, транзакция, получающая данные от пользователя) — элементарный процесс, перемещающий данные из внешней среды в приложение. Данные могут поступать с экрана ввода или из другого приложения. Данные могут содержать как управляющую, так и деловую информацию. Обработываемые данные могут соответствовать одному или нескольким внутренним логическим файлам
- **Внешний вывод** (EO, транзакция передающая данные пользователю) — элементарный процесс, перемещающий данные, вычисленные в приложении, во внешнюю среду и связанный с созданием и/или обработкой выходной информации приложения — выходного отчета, документа, экранной формы
- **Внешний запрос** (EQ, интерактивный диалог с пользователем, требующий от него каких-либо действий) — элементарный процесс, состоящий из комбинации «запрос/ответ», не связанный с вычислением производных данных или обновлением внутренних логических файлов (базы данных)
- **Внутренний логический файл** (ILF, информация, которая используется во внутренних взаимодействиях системы) — выделяемые пользователем логически связанные группы данных или блоки управляющей информации, которые поддерживаются внутри продукта и обслуживаются через внешние вводы
- **Внешний интерфейсный файл** (EIF, файлы, участвующие во внешних взаимодействиях с другими системами) — выделяемые пользователем логически связанные группы данных или блоки управляющей информации, на которые ссылается продукт, но которые поддерживаются вне продукта

Ранг и оценка сложности внешних вводов

Ссылки на файлы (FTR)	Элементы данных (DET)		
	1-4	5-15	>15
0-1	Низкий (3)	Низкий (3)	Средний (4)
2	Низкий (3)	Средний (4)	Высокий (6)
>2	Средний (4)	Высокий (6)	Высокий (6)

Ранг и оценка сложности внешних выводов

Ссылки на файлы (FTR)	Элементы данных (DET)		
	1-4	5-19	>19
0-1	Низкий (4)	Низкий (4)	Средний (5)
2-3	Низкий (4)	Средний (5)	Высокий (7)
>3	Средний (5)	Высокий (7)	Высокий (7)

Ранг и оценка сложности внешних запросов

Ссылки на файлы (FTR)	Элементы данных		
	1-4	5-19	>19
0-1	Низкий (3)	Низкий (3)	Средний (4)
2-3	Низкий (3)	Средний (4)	Высокий (6)
>3	Средний (4)	Высокий (6)	Высокий (6)

Ранг и оценка сложности внутренних логических файлов

Типы элементов-записей (RET)	Элементы данных (DET)		
	1-19	20-50	>50
1	Низкий (7)	Низкий (7)	Средний (10)
2-5	Низкий (7)	Средний (10)	Высокий (15)
>5	Средний (10)	Высокий (15)	Высокий (15)

Ранг и оценка сложности внешних интерфейсных файлов

Типы элементов-записей (RET)	Элементы данных (DET)		
	1-19	20-50	>50
1	Низкий (5)	Низкий (5)	Средний (7)
2-5	Низкий (5)	Средний (7)	Высокий (10)
>5	Средний (7)	Высокий (10)	Высокий (10)

Для оценки сложности используются следующие характеристики:

- **RET (тип элемента записи)** – самая большая идентифицируемая пользователем логическая группа данных внутри ILF или EIF
- **DET (тип элемента данных)** – это уникальное поле данных
- **FTR (файл типа ссылки)** – сколько разных файлов / таблиц (ILF или EIF) затрагивает одна транзакция (читает или меняет).

Порядок расчета трудоемкости разработки ПО:

1. Определение количества функциональных типов данных для приложения

2. Определение количества связанных с каждым функциональным типом элементарных данных (DET), элементарных записей (RET) и файлов типа ссылок (FTR)
3. Определение сложности (в зависимости от количества DET, RET и FTR)
4. Подсчет количества функциональных точек приложения
5. Подсчет количества функциональных точек с учетом общих характеристик системы
6. Оценка трудоемкости разработки (с использованием различных статистических данных)

Скорректированное количество функциональных точек:

$FP = \text{Общее количество} * (0,65 + 0,01 * \sum Fi)$,

где Fi — 14 коэффициентов регулирования сложности, от 0 до 5

№	Системный параметр	Описание
1	Передача данных	Сколько средств связи требуется для передачи или обмена информацией с приложением или системой?
2	Распределенная обработка данных	Как обрабатываются распределенные данные и функции обработки?
3	Производительность	Нуждается ли пользователь в фиксации времени ответа или производительности?
4	Эксплуатационные ограничения	Насколько сильны эксплуатационные ограничения и каков объем специальных усилий на их преодоление?
5	Частота транзакций	Как часто выполняются транзакции (каждый день, каждую неделю, каждый месяц)?
6	Оперативный ввод данных	Какой процент информации надо вводить в режиме онлайн?
7	Эффективность работы конечных пользователей	Приложение проектировалось для обеспечения эффективной работы конечного пользователя?
8	Оперативное обновление	Как много внутренних файлов обновляется в онлайн-транзакции?
9	Сложность обработки	Выполняет ли приложение интенсивную логическую или математическую обработку?
10	Повторная используемость	Приложение разрабатывалось для удовлетворения требований одного или многих пользователей?
11	Легкость инсталляции	Насколько трудны преобразование и инсталляция приложения?
12	Легкость эксплуатации	Насколько эффективны и/или автоматизированы процедуры запуска, резервирования и восстановления?
13	Количество возможных установок на различных платформах	Была ли спроектирована, разработана и поддержана возможность инсталляции приложения в разных местах для различных организаций?
14	Простота изменений (гибкость)	Была ли спроектирована, разработана и поддержана в приложении простота изменений?

Трудоемкость (трудозатраты) = $FP * P$

где: FP — количество функциональных точек; P — средние затраты времени на реализацию одной точки (индекс производительности)

Индекс производительности индивидуален для каждой команды. Ниже приведены усредненные рыночные данные (в среднем по индустрии):

- Высокая производительность: 4–7 чел.-часов на 1 FP (опытная команда, готовые фреймворки)
- Средняя производительность: 8–12 чел.-часов на 1 FP (стандарт для большинства проектов)

- Низкая производительность: 15–20+ чел.-часов на 1 FP (сложные системы, жесткое регулирование, новички в разработке)

Решение

Авторизация

Внешний ввод (EI)

- DET = 4 уникальных поля
- FTR = 1 таблицу читает при логине

Внешний вывод (EO)

- нет экспорта

Внешний запрос (EQ)

- нет диалога с пользователем

Внутренний логический файл (ILF)

- DET = 2 поля описывают человека
- RET = 1 таблица

Внешний интерфейсный файл (EIF)

- нет импорта

Биржевые сводки

Внешний ввод (EI) - добавление кнопки

- DET = 3
- FTR = 1

Внешний вывод (EO) - таблица со сводками

- DET = 3 колонки
- FTR = 1

Внешний запрос (EQ)

- нет диалога с пользователем

Внутренний логический файл (ILF) - таблица со сводками в БД

- DET = 3 поля описывают сводку
- RET = 1 таблица

Внешний интерфейсный файл (EIF)

- DET = 3 поля описывают сводку

- RET = 1 таблица

Заявки

Внешний ввод (EI) - удаление и изменение заявки

- DET = 8 поля взаимодействия с 2 действиями
- FTR = 1

Внешний вывод (EO) - таблица с заявками

- DET = 4 колонки
- FTR = 1

Внешний запрос (EQ)

- нет диалога с пользователем

Внутренний логический файл (ILF) - таблица с заявками в БД

- DET = 4 поля описывают заявку
- RET = 1 таблица

Внешний интерфейсный файл (EIF)

- нет импорта

Новая заявка

Внешний ввод (EI) - удаление и изменение заявки

- DET = 5
- FTR = 1

Внешний вывод (EO) - таблица с заявками

- нет вывода

Внешний запрос (EQ)

- нет диалога с пользователем

Внутренний логический файл (ILF) - таблица с заявками в БД

- DET = 4 поля описывают заявку
- RET = 1 таблица

Внешний интерфейсный файл (EIF)

- DET = 4 поля описывают заявку
- RET = 1 таблица

Итого

— ILF: низкий – 4

- EIF: низкий – 1
- EI: низкий – 3, средний – 2
- EO: низкий – 2
- EQ: низкий – 1

С использованием разработанного калькулятора, внося указанные элементы, характеристики проекта и используемые языки получились следующие значения:

Метод функциональных точек

Параметр	Просто		Средне		Сложно		Итого
	Количество	Коэффициент	Количество	Коэффициент	Количество	Коэффициент	
Внешние входы (EI)	3	3	2	6	0	4	0
Внешние выходы (EO)	2	4	0	5	0	7	0
Внешние запросы (EQ)	1	3	0	4	0	6	0
Внутренние логические файлы (ILF)	4	7	0	10	0	15	0
Внешние логические файлы (EIF)	1	5	0	7	0	10	0
Общее количество							0

Системные параметры приложения

Передача данных	5	Оперативное обслуживание	4
Распределенная обработка данных	5	Сложность обработки	4
Производительность	3	Платформа использования	0
Высотные ограничения	2	Легкость установки	1
Частота транзакций	3	Легкость эксплуатации	2
Оперативный ввод данных	4	Количество возможных установок на различных платформах	2
Эффективность работы конечных пользователей	1	Простота изменений (гибкость)	2

Язык программирования

SQL	- 15	%
C#	- 60	%
Java	- 25	%

Добавить язык

Рассчитать

 fp = 65, corrected fp = 66.95, loc = 3492



COCOMO II

COCOMO (Constructive COst MOdel — конструктивная стоимостная модель) — алгоритмическая модель оценки стоимости разработки программного обеспечения, разработанная Барри Боэмом. Цель модели — оценить трудоёмкость, время и стоимость проекта, учитывая сложность проекта и другие факторы.

Три различные модели оценки стоимости в COCOMO II:

- **Модель композиции приложения** – это модель, которая подходит для проектов, созданных с помощью современных инструментальных средств. Единицей измерения служит объектная точка
- **Модель ранней разработки архитектуры**. Эта модель применяется для получения приблизительных оценок проектных затрат периода выполнения проекта перед тем как будет определена архитектура в целом. В этом случае используется небольшой набор новых драйверов затрат и новых уравнений оценки. В качестве единиц измерения используются функциональные точки либо KSLOC
- **Постархитектурная модель** – наиболее детализированная модель COCOMO II, которая используется после разработки архитектуры проекта. В состав этой модели включены новые драйверы затрат, новые правила подсчета строк кода, а также новые уравнения

Модель композиции приложения	Модель ранней разработки архитектуры	Постархитектурная модель
Грубые входные данные	Ясно понимаемые особенности проекта	Детальное описание проекта
Оценки низкой точности	Оценки умеренной точности	Высокоточные оценки
Приблизительные требования	Ясно понимаемые требования	Стабилизировавшиеся основные требования
Концепция архитектуры	Ясно понимаемая архитектура	Стабильная базовая архитектура

Модель композиции приложения

Модель композиции приложения -- это модель, которая подходит для проектов, созданных с помощью современных инструментальных средств. Единицей измерения служит объектная точка.

- Грубые входные данные
- Оценки низкой точности
- Приблизительные требования

- Концепция архитектуры

Данная модель используется на ранней стадии конструирования ПО

Объектная точка — средство косвенного измерения ПО. Подсчет количества объектных точек производится с учетом количества экранов (как элементов пользовательского интерфейса), отчетов и компонентов, требуемых для построения приложения

Правила подсчета объектных точек:

- Количество изображений на дисплее (экранных форм). Простые изображения принимаются за 1 объектную точку, изображения умеренной сложности принимаются за 2 точки, очень сложные изображения принято считать за 3 точки
- Количество представленных отчетов. Для простых отчетов назначаются 2 объектные точки, умеренно сложным отчетам назначаются 5 точек. Написание сложных отчетов оценивается в 8 точек
- Количество модулей, которые написаны на языках третьего поколения (C, Java, Python и пр.) и разработаны в дополнение к коду, написанному на языке программирования четвертого поколения. Каждый модуль на языке третьего поколения считается за 10 объектных точек

Формула для расчёта трудозатрат и времени разработки:

$NOP = (\text{Объектные точки}) \times [(100 - \%RUSE) / 100]$ – новые объектные точки

$ТРУДОЗАТРАТЫ = NOP / PROD$ [чел.- мес.]

PROD - оценка скорости разработки

где: RUSE – процент переиспользуемого кода

$Время = 3,67 * (Трудозатраты)^{(0.28 + 0.2 * (p-1.01))}$

Решение

Расчёт объектных точек

Экранные формы:

- Авторизация – средняя сложность
- Биржевые сводки – средняя сложность
- Новая заявка – простая сложность
- Заявки – простая сложность

Отчёты:

- Список сводок – средняя сложность
- Список заявок – простая сложность

Модули:

- Java – 25%
- C# – 60%
- SQL – 15%

Фактор	Уровень	Обоснование
PREC — прецедентность	наличие некоторого числа прецедентов	Предметная область «относительно новая», лишь у отдельных членов команды есть опыт подобных систем
FLEX — гибкость разработки	большой частью согласованный процесс	Заказчик не требует жёсткой регламентации процесса (ограничение — только жёсткий график)
RESL — разрешение рисков	некоторое (40%)	«Анализу архитектурных рисков уделено лишь некоторое внимание»
TEAM — сплочённость команды	некоторая согласованность	Команда новая, но проведены мероприятия по сплочению, коммуникация «приемлемая»
PMAT — зрелость процессов	1+CMM	Организация «только начинает внедрять» методы управления проектами и оценку качества

Среднюю зарплату работника примем как 80000 у.е. в месяц.

Опытность коадны: низкая

Модель композиции приложения		Результат	
%RUSE	0	Параметр П	1.2193
Опытность команды/разработчика	Низкая	Количество окон	6
Экранные формы Простые: 2 Умеренные: 2 Сложные: 0		Трудозатраты	6.143 человеко-месяцев
Отчеты Простые: 1 Умеренные: 1 Сложные: 0		Длительность	5.892 месяцев
Модули на языках 3 поколения	3	Средняя численность команды разработчиков	1 человек
Рассчитать		Бюджет	471360.0 рублей

Модель ранней разработки архитектуры.

Эта модель применяется для получения приблизительных оценок проектных затрат периода выполнения проекта перед тем как будет определена архитектура в целом

- Ясно понимаемые особенности проекта
- Оценки умеренной точности
- Ясно понимаемые требования
- Ясно понимаемая архитектура

Формулы:

Трудозатраты = $2,94 * E_{Arch} * (Размер)^p$,

где:

Трудозатраты (работа) — число человеко-месяцев

$E_{Arch} = PERS * RCPX * RUSE * PDIF * PREX * FCIL * SCED$

Размер — KSLOC (предпочтительно для подсчета KSLOC предварительно подсчитать количество функциональных точек)

p — показатель степени

Время = $3,67 * (Трудозатраты)^{(0.28 + 0.2 * (p-1.01))}$

Множитель E_{Arch} является произведением семи показателей, характеризующих проект и процесс создания ПО, а именно:

- PERS — квалификация персонала (Сверхнизкий — аналитики и программисты имеют низшую квалификацию, текучесть больше 45%; Сверхвысокий —

аналитики и программисты имеют высшую квалификацию, текучесть меньше 4%)

- RCPX — сложность и надежность продукта (Сверхнизкий — продукт простой, специальных требований по надежности нет, БД маленькая, документация не требуется; Сверхвысокий — продукт очень сложный, требования по надежности жесткие, БД сверхбольшая, документация требуется в полном объеме)
- RUSE — разработка для повторного использования (Низкий — не требуется; Сверхвысокий — требуется переиспользование в других продуктах)
- PDIF — сложность платформы разработки (Сверхнизкий — специальные ограничения по памяти и быстродействию отсутствуют, платформа стабильна; Сверхвысокий — жесткие ограничения по памяти и быстродействию, платформа нестабильна)
- PREX — опыт персонала (Сверхнизкий — новое приложение, инструменты и платформа; Сверхвысокий — приложение, инструменты и платформа хорошо известны)
- FCIL — оборудование (Сверхнизкий — инструменты простейшие, коммуникации затруднены; Сверхвысокий — интегрированные средства поддержки жизненного цикла, интерактивные мультимедиа коммуникации)
- SCED — сжатие расписания (Очень низкий — 75% от номинальной длительности; Очень высокий — 160% от номинальной длительности)

Факторы:

Драйвер	Уровень	Обоснование из текста задачи
PERS	номинальный	«Возможности персонала (PERS) – средние»
RCPX	очень высокий	«Надёжность и уровень сложности (RCPX) разрабатываемой системы оцениваются как очень высокие»
RUSE	номинальный	«повторного использования компонентов не предусматривается (RUSE)» — базовый уровень без требований к reuse
PDIF	высокий	«Сложность платформы (PDIF) – высокая»
PREX	низкий	«опыт работы в разработке систем подобного типа (PREX) низкий»
FCIL	очень высокий	«Разработка предусматривает очень интенсивное использование инструментальных средств поддержки (FCIL)»
SCED	низкий	«Заказчик настаивает на жестком графике» — сжатие сроков разработки

Модель ранней разработки архитектуры		Результат	
Сложность продукта (RCPX)	Очень высокий ▾	Параметр П	1.2193
Необходимость повторного использования (RUSE)	Номинальный ▾	Трудозатраты	13.552 человеко-месяцев
Сложность платформы (PDIF)	Высокий ▾	Длительность	7.908 месяцев
Опытность персонала (PREX)	Низкий ▾	Средняя численность команды разработчиков	2 человек
Способности персонала (PERS)	Номинальный ▾	Бюджет	632640.0 рублей
Возможности среды (FCIL)	Очень высокий ▾		
Сроки (SCED)	Низкий ▾		
Рассчитать			

Заключение

В ходе выполнения лабораторной работы были изучены основные подходы к предварительной оценке параметров программных проектов с использованием методики COCOMO II. Практическая часть выполнялась на основе модели композиции приложения и модели ранней разработки архитектуры.

В рамках модели композиции приложения проведён подсчёт объектных точек с учётом сложности экранов и отчётов, а также процентного соотношения языков программирования. С учётом масштабных факторов проекта (PREC, FLEX, RESL, TEAM, PMAT) рассчитаны трудозатраты, срок разработки и бюджет. В модели ранней разработки архитектуры на основе KSLOC и семи драйверов трудоёмкости получены более консервативные оценки.

Сравнение результатов показало, что модель ранней разработки архитектуры даёт более высокие оценки благодаря учёту большего числа факторов. Применение COCOMO II на ранних этапах позволяет обоснованно планировать стоимость, сроки и ресурсы проекта.